COMPUTER NETWORKS PROJECT · SPEAKING GUIDE

How to present the AI-Adaptive UDP File Transfer project

Go through the slides in order. For each one: what to show, what to say in plain words, the one number to remember, and the question a faculty member is most likely to ask.

Before you start

- [ ] Open the deck `UDP_ML_Timeouts_Presentation.pptx` and switch to **Presenter View**. Your speaker notes are already on every slide.
- [ ] Open Terminal, go to the project folder and start the lab: `cd ~/Documents/udp_ai_transfer` then `python3 lab_server.py`
- [ ] Open `http://localhost:8000` in the browser and scroll to the **Live lab**. Keep this tab ready for the demo.
- [ ] Do one practice run in the lab so the first real run is not slow.
- [ ] Remember three numbers: **420** transfers, **+4.3 %** faster, **12 %** less waste.

The 30-second version

Say this if someone asks “what does your project do?” or if you only get one minute.

UDP sends packets but never checks if they arrive. We built a reliable file transfer on top of UDP, like a mini TCP. The hard part is deciding how long to wait before resending a packet. Wait too little and you resend packets that were not lost. Wait too long and the transfer sits idle. TCP uses a fixed formula for this. We trained machine-learning models to choose the wait time instead, and tested everything on a simulated bad network: 420 transfers, every file verified. Our best version, ML plus a smart loss detector called RACK, was 4.3 % faster than TCP's formula and wasted 12 % fewer packets.

Slide by slide

1–3 The problem

4–7 What we built

8–10 The ML and RACK

11–15 Results

16–18 Wrap up

Full talk: all 18 slides, about 10 minutes.

Slide 1 Title

20 s

SHOW Title slide.

SAY "Good morning. Our project is a reliable file transfer protocol built on UDP. We tested whether machine learning can decide when to resend a lost packet better than the formula TCP uses."

Slide 2 The problem

50 s

SHOW The RTT histogram. Point at the long tail on the right.

SAY “UDP does not resend lost packets, so we have to. The question is how long to wait for an acknowledgement. Too short, and we resend packets that were only late. That is waste. Too long, and we sit idle. That is slow. This chart is the round-trip time on one network. Most packets come back in about 65 ms, but some take much longer. So the right waiting time keeps changing.”

NUMBER `median 64.8 ms · 95th percentile 78.9 ms`

If asked: what is RTT?

Round-trip time. The time from sending a packet to getting its acknowledgement back.

Slide 3 What we did

30 s

SHOW The contributions list.

SAY “We did four things. One, built the reliable protocol. Two, built a network simulator that damages packets the same way every time, so the comparison is fair. Three, trained two ML timeout models. Four, added a smarter loss detector called RACK, and tested all of it in 420 transfers.”

Slide 4 Architecture

40 s

SHOW Sender → proxy → receiver diagram.

SAY “There are three programs. The sender splits the file into packets. The receiver puts them back together. In the middle is our proxy. It acts like a bad internet connection: it drops, delays, duplicates and reorders packets. Because it uses a seed, the same seed causes exactly the same damage. So every strategy faces the identical network.”

If asked: why a simulator and not the real internet?

The real internet changes every second, so two runs are never comparable. The simulator lets us repeat the exact same conditions for every strategy.

Slide 5 Packet header SKIP IF SHORT

30 s

SHOW The 24-byte header diagram.

SAY “Every packet has a 24-byte header with a type, a sequence number and a CRC-32 checksum to catch corrupted packets. The whole file is checked with SHA-256 at the end. A packet is at most 1,268 bytes on the wire. That is below 1,280, the smallest size IPv6 must carry, so packets are never split.”

NUMBER `24-byte header · 1,200-byte payload`

Slide 6 Selective Repeat

40 s

SHOW The sender/receiver ladder diagram.

SAY “We use Selective Repeat, from the CN syllabus. The sender can have 64 packets in flight. If one is lost, only that one is resent, not everything after it. The receiver stores out-of-order packets until the gap is filled.”

NUMBER `window = 64 packets`

If asked: why not Go-Back-N?

Go-Back-N resends the whole window after one loss. On a lossy network that wastes a lot. Selective Repeat resends only what was lost.

Slide 7 TCP’s timeout formula SKIP IF SHORT

40 s

SHOW The RFC 6298 equations.

SAY “This is the baseline: the formula TCP uses, from RFC 6298. Timeout equals the smoothed RTT plus 4 times the variation. If a timer fires, the timeout doubles. While building it we found a bug in our backoff: old timers kept raising the wait. Fixing it made one transfer go from 22 seconds to 4.3 seconds.”

NUMBER `RTO = SRTT + 4 × RTTVAR`

If asked: what is Karn’s algorithm?

Do not measure RTT on a resent packet, because you cannot tell which copy the acknowledgement belongs to.

Slide 8 The ML model

50 s

SHOW The ML pipeline diagram.

SAY “Instead of a fixed formula, the model looks at the last few RTTs: average, spread, maximum, trend and so on. That is 12 features. It predicts how long the slowest 5 % of the next packets will take, and uses that as the timeout. Version 1 is a random forest with 40 trees. For safety, the answer is always kept between half and twice the TCP value. If anything goes wrong, it falls back to the formula.”

NUMBER `12 features · 40 trees · safety band 0.5× to 2×`

If asked: why save the model as JSON and not pickle?

Loading a pickle file can run code, which is a security risk. JSON is only data. The trees are evaluated in plain Python.

Slide 9 Version 2

40 s

SHOW The v2 calibration chart.

SAY “Version 1 was faster, but it resent about 2.5 times more packets than needed. So we built version 2. It uses gradient boosting with 300 trees, and it was also trained on data recorded while ML was in control. We fixed the rule before testing: waste must stay under 1.15 times TCP’s, and within that, pick the fastest setting.”

NUMBER `v1 waste 2.47× → v2 waste 0.71×`

Slide 10 RACK loss detection

40 s

SHOW The RACK slide.

SAY “A timer is not the only way to find a lost packet. RACK, from RFC 8985, says: if a packet sent later has already been acknowledged, and enough time has passed, the earlier one is probably lost. So we resend without waiting for the timer. A simple duplicate-ACK rule gave us about 3,400 false resends. RACK with an adaptive reorder window brought that down to about 6.”

NUMBER `~3,400 false resends → ~6`

Slide 11 How we tested SKIP IF SHORT

40 s

SHOW Method slide and the networks table.

SAY “Seven networks, from stable to heavy jitter to reordering. Ten seeds each. Six strategies. That is 420 transfers of a 1 MB file. Every file arrived and matched its SHA-256 hash. Each strategy is compared with TCP’s formula on the same seed, and we use the Wilcoxon signed-rank test to check the difference is not luck.”

NUMBER `7 × 10 × 6 = 420 transfers, 420 verified`

Slide 12 Headline result

40 s

SHOW The dark headline-stats slide. Pause here.

SAY “This is the main result. ML version 2 together with RACK was 4.3 % faster than TCP’s formula on average and resent 12 % fewer unnecessary packets. It is the only strategy that is better on both speed and waste.”

NUMBER `+4.3 % goodput · 0.88× waste`

If asked: what is goodput?

Useful file data delivered per second. It does not count resent copies or headers.

Slide 13 Speed vs waste

40 s

SHOW The scatter chart. Point at each group.

SAY “Up and to the left is better. ML v1 and a shorter TCP formula are fast but wasteful. ML v2 alone is lean but a bit slow. RACK alone is fast with some extra waste. The combination sits in the best corner.”

Slide 14 Per network

40 s

SHOW The per-network bar chart.

SAY “ML v2 plus RACK was never slower than TCP’s formula on any lossy network. The biggest gain was on the reordering network, 10.2 %. The gains on lossy, jitter and reordering networks are statistically significant.”

NUMBER `never slower · up to +10.2 % · significant on 3 of 6`

If asked: what about the stable network?

About 2 % slower there. With no losses, the timeout never matters, and the extra Python computation costs a little time. A compiled version would remove most of it.

Slide 15 **The lesson** 40 s

SHOW Offline vs online slide.

SAY “Our most important lesson. On recorded data, ML v1 predicted much better than the formula: 5.7 ms error against 10.0. But in real transfers it resent more packets. Being accurate on average is not the same as being a better protocol. That is why we chose version 2 using full transfers, not prediction error.”

NUMBER `error 5.7 ms vs 10.0 ms, yet more waste`

Slide 16 **Engineering + live demo** 60 s

SHOW The engineering slide, then switch to the browser for the demo below.

SAY “The project has 136 automated tests, all passing. We also built a live lab. Let me show you a real transfer.”

NUMBER `136 tests`

Slide 17 **Conclusion** 30 s

SHOW The dark conclusion slide.

SAY “To conclude: ML alone is a trade-off between speed and waste. RACK, a protocol idea with no ML, is the most dependable gain. Together they beat TCP’s formula on both counts. Our limits: it is a simulator on one machine, and there is no congestion control.”

Slide 18 **Thank you** 10 s

SAY “Thank you. We are happy to take questions.”

Live demo, step by step

1. Switch to the browser tab at `localhost:8000` and scroll to **Live lab**.
2. Choose network **jitter_heavy**, strategy **RFC 6298**, RACK **off**. Press run. Say: “This is TCP’s formula on a jittery network.”
3. While it runs, point at the timeouts and resends counters as they rise.
4. Now choose **ML v2** with RACK **on**, same network. Press run. Say: “Same network, same damage, our best strategy.”
5. Compare the two: time taken, and the number of unnecessary resends. Then say: “Both files arrived and were verified with SHA-256.”

If the lab does not start, the page replays recorded runs automatically. You can still show it the same way.

```
cd ~/Documents/udp_ai_transfer
python3 lab_server.py      # then open http://localhost:8000
```

Likely questions

▼ Why use UDP and not just TCP?

TCP hides its timeout inside the operating system, so we cannot change it. On UDP we build reliability ourselves and can swap the timeout strategy. QUIC, used by HTTP/3, also runs on UDP for the same reason.

▼ Does the ML always win?

No. ML alone either wastes more (v1) or is slower (v2). It only wins on both counts when combined with RACK. We report this honestly.

▼ What if the model gives a bad answer?

The timeout is clamped between half and twice the TCP value, with a floor from recent RTTs. If the model fails or has too little data, the sender uses the TCP formula.

▼ Why predict the 95th percentile?

The timeout should cover almost all packets, but not wait for the rare slowest one. The 95th percentile is that balance.

▼ How do you know the comparison is fair?

The simulator decides each packet's fate from a hash of the seed and packet number. Same seed gives same damage. Every strategy runs on the same 70 network-and-seed pairs.

▼ Is the result statistically significant?

We use the Wilcoxon signed-rank test on paired seeds. ML v2 plus RACK is significantly faster on 3 of 6 lossy networks, and never slower on any.

▼ Would this work on the real internet?

Not proven. It is a simulator on one machine with no congestion control. Testing on real links is future work.

▼ What did you find hardest?

The backoff bug that made a transfer take 22 seconds instead of 4.3, and learning that better prediction accuracy did not mean a better protocol.

Speaking tips

One idea per slide
Say the main point first, then the detail. If you forget the detail, the main point is enough.

Pause on slide 12
That is your result. Let the numbers sit for two seconds before moving on.

Be honest about limits
Saying "ML alone did not win" shows you understand the work. Faculty like that.

Unknown question?
Say: "Good question. We did not test that, but I would expect..." and give your best reasoning.

Numbers are from the project’s 420-transfer run (results/RESULTS.md).